Итоговый протокол сравнения

Тройное сравнение генераторов цифровых двойников / DES-моделей на основе LLM

Методы	**A** = FactoryFlow (`FactoryFlow/FactoryFlow_v2`) · **B** = LLM-DES-Modeling · **C** = DigitalTwinBuilder (сервис)
Стенд	`/ssd/projects/dt-eval/comparison` (`run_comparison.py`, `adapters/`, `report.py`, `truth/`)
Дизайн	5 синтетических последовательных линий × 3 метода × 3 повтора, всё в `mode: real`
Статус	итоговый протокол, 2026-09-25
Сопутствующий документ	`protocol.md` — рабочий протокол с журналом изменений «только на добавление» (§8). Настоящий документ — самодостаточное резюме для передачи; при расхождениях аудиторским следом служит журнал изменений `protocol.md`.

Документ содержит по порядку: **§1** — метод, которым проводилось сравнение; **§2–§3** — краткие описания методов A и B; **§4** — DES-этап внутри DT-агента метода C; **§5** — таблицу итогового прогона для A, B и C (и оговорки, которые читатель обязан к ней применить); **§6** — угрозы валидности; **§7** — воспроизводимость.

---

1. Метод сравнения

1.1 Вопрос сравнения

> Для **одной и той же базовой системы** (синтетическая установка с известными параметрами) — какой метод порождает имитационную модель, чьё **поведение** (пропускная способность, WIP, энергия) точнее всего совпадает с реальной установкой, насколько **устойчиво** по повторным генерациям, насколько **дёшево** и с какой **структурной точностью**?

1.2 Объекты сравнения

Метод	Инструмент	Собственный вход, который он потребляет	Выход, который он производит
A	FactoryFlow / FactoryFlow_v2	Описание линии на естественном языке (проза)	Исходный код Python, задающий сеть компонентов FactorySimPy
B	LLM-DES-Modeling	CSV-журнал событий установки (process mining через pm4py)	Исходный код Python самодостаточной модели SimPy
C	DigitalTwinBuilder (удалённый сервис FastAPI)	**Интервью** (метод задаёт вопросы; стенд отвечает из того же набора сведений)	JSON требований → схема PostgreSQL (агент 1) и, через агента 2, конфигурация двойника, скрипт PyChrono и **дискретно-событийная модель SimPy**

Каждый метод используется **так, как он опубликован** — стенд меняет только оркестрацию (вызов процесса, преобразование ввода-вывода, разбор результата). Там, где обнаруживался дефект в собственном контракте вывода метода, он исправлялся **в репозитории этого метода**, а не обходился в стенде (см. §4.6 о том, как исправления C попадают в измерение, не будучи перепечатанными).

1.3 Правила честности

1. **Собственные входы.** Каждый метод получает вход, для которого его проектировали авторы. Каждый набор кейса несёт входные данные по методам, описывающие **одну и ту же** установку.
2. **Однократная генерация.** Один проход генерации на метод на кейс (A: `run_amg_wrapper`; B: `ModelBuilder.build`). Замкнутый контур (`run_manual_overrides`, `Modeloptimizer`) не используется.

3. **Никаких перебросов.** Неудачная генерация фиксируется как неудача и никогда не переигрывается ради лучшего образца. **Доля успеха сама является метрикой.**
4. **Сидирование.** Эталонная имитация, каждый прогон модели и каждый управляемый стохастический элемент сидируются от одного базового seed уровня кейса. Выборка LLM **не** пришпилена температурой, поэтому повтор измеряет подлинную дисперсию генератора.
5. **Автономность.** Вопросы, которые метод задал бы человеку, стенд детерминированно отвечает из информационного набора кейса, а число ходов оценивается как стоимость автономности.
6. **Изоляция.** Каждый метод работает в своём venv и рабочем каталоге; побочные эффекты остаются внутри рабочего каталога прогона.
7. **Фиксируется, а не примиряется.** Противоречивые или деградировавшие исходы сообщаются как наблюдаемые (по каждому повтору, никогда не объединяются) и не сглаживаются в среднее, которого не дал ни один отдельный прогон.

1.4 Набор кейсов и эталон

Пять неизменяемых наборов кейсов в `cases/`, по одному на установку. Каждый содержит `manifest.yaml` / `manifest.json` (спецификация эталона), `eventlog.csv` (вход B), `prompt_A.txt` (вход A), `aux_B.txt`, `truth_kpis.json`, `station_table.md`, `truth_structure.json`. Все порождаемые артефакты детерминированно создаются `casegen.py` и закоммичены, поэтому все методы всегда потребляют побайтово идентичные входы.

Эталонный движок — `truth/plant.py`, чистый SimPy-завод, не зависящий от библиотек обоих инструментов. Усредняется `reference.replications = 10` сидированных прогонов. Окно отказа соблюдается только пока станок свободен (никогда в середине обработки), что сохраняет равенство строк журнала `Working` окнам обработки одной детали.

Кейс	Эталонная пропускная способность (дет./ч)	Эталонный WIP (дет.)	Эталонная энергия (кВт·ч/дет.)
`case_0001_serial_line`	59.868	40.387	0.9892
`case_0002_saturated_short_line`	30.620	27.917	0.3525
`case_0003_starved_six_line`	11.091	5.251	2.3537

Кейс	Эталонная пропускная способность (дет./ч)	Эталонный WIP (дет.)	Эталонная энергия (кВт·ч/дет.)
	Эталонная пропускная способность (дет./ч)	Эталонный WIP (дет.)	Эталонная энергия (кВт·ч/дет.)
	лодание		
case_0004_failure_heavy_line	22.401 частыми отказами	7.289	0.8735
case_0005_high_rate_line	44.991 высокая интенсивность поступлений	38.882	0.3354

Определения: пропускная способность = завершённые детали / часы окна; WIP = среднее по времени число деталей в системе; энергия на деталь = (Σ работа · work_power + Σ простой · idle_power) / 3600 кВт·ч за окно ÷ завершённые детали. Единицы: дет./ч, дет., кВт·ч/дет.; все времена в секундах.

Собственный вход C — интервью, а не файл. UI-агент C задаёт вопросы, и стенд отвечает LLM-«инженером установки», в системный промпт которого заложены `prompt_A.txt` плюс точные значения параметров из манифеста — тот же информационный набор, который A получает прозой, а B — журналом. Поскольку UI-промпт C — это протокол интервью, анализирующий сообщение **пользователя**, стенд отправляет вступительное сообщение инженера первым ходом пользователя перед подачей задачи. Фиксированная побайтово идентичная строка политики отвечает на вопросы, на которые не может ответить ни один набор (для чего нужен двойник, как часто обновляются данные, что пороги сигнализации не определены). Она не содержит значений параметров и не оценивается; на любой другой недостающий факт следует отказ. Каждый ход пишется в `artifacts/interview.log`, а число ходов — это стоимость автономности C.

1.5 Конвейер и единый контракт вывода

На кейс × метод × повтор: **сгенерировать → выполнить → разобрать → оценить.**

Каждый адаптер — самостоятельный CLI, запускаемый интерпретатором своего метода, и пишет один и тот же `result.json`: `method`, `case`, `rep`, `status`, `error`, `elapsed_generation_s`, `elapsed_simulation_s`, `llm_calls`, `llm_tokens_estimate`, `generated_model_file`, `kpis` (`throughput_per_hour`, `utilization`, `wip_parts`, `energy_per_part_kwh`, `native`), `assumptions`, `notes`.

Отсутствующий или нечисловой показатель — это `null`, **никогда не выдумка**. Методы, не способные выразить показатель, оставляют его null; сама асимметрия покрытия — это находка (выразительность модели), а не ошибка.

Матрица покрытия

Показатель	Эталон	A	B	C
пропускная способность	✔	✔	✔	✔ (слот DES, исполняется)
загрузка по каждому станку	✔	✔	—	—
WIP	✔	приблизительно (буферы)	✔	✔ (слот DES, исполняется)
энергия на деталь	✔	—	✔	✔ (слот DES, исполняется)

Показатель	Эталон	A	B	C
восстановленная структура	✔	✔	✔	✔ (требования + схема; конфигурация/скрипт двойника захвачены, но не исполняются; DES исполняется)

На практике выходной KPI у A — это пропускная способность (плюс отдельно сообщаемая APE загрузки); WIP и энергию он не выражает вовсе.

1.6 Семантика статусов

`ok | service_unavailable | generation_error | simulation_error | timeout | parse_error`

- **Сбои самого стенда** (его LLM-ответчик в OpenRouter недоступен) — это `service_unavailable` и вина зависимости, а не метода: проверка доступности C, чтения, опросы и создание сессии повторяются (`C.net_retries`) до этого вердикта. `HTTPError` никогда не `service_unavailable`: сервис ответил, значит это засчитывается методу.
- **Статус повтора C выводится из его этапов**, а не из «конвейер вернул результат». Повтор C — `ok`, только если его слот DES произвёл несущий контракт KPI-нагрузку; иначе вердикт берётся от самого глубокого отказавшего этапа (исполнение завершилось с ненулевым кодом / таймаут / нет контракта / нет модели / таймаут опроса этапа), и `native.status_basis` фиксирует, от какого этапа он пришёл. Это правило живёт в `adapters/base.py::derive_rep_status` и применяется и адаптером, и `report.py`, поэтому повтор, сохранённый по старой семантике, **отображается** по исходу его этапа с отметкой `(recorded ok)` рядом, а не молча переписывается.

1.7 Измеряемые размерности и формулы

На кейс `c`, метод `m`, повтор `r ∈ 1..R`:

Размерность	Метрика	Формула
Валидность	доля успеха	доля генераций, давших исполнимую модель и разбираемые KPI
Поведение	MAPE по показателю	на повтор — среднее `|pred−эталон|/эталон` по показателям, которые выражают **обе** стороны; и отдельно по каждому показателю
Поведение	тест распределения	2-выборочный KS между значениями KPI метода по его повторам и эталонными по его референсным повторениям
Структура (вторичная)	ошибка параметра	для каждого параметра в `truth_structure.json` — `|pred−истина|/истина`; извлечение зависит от метода и делается по мере сил
Устойчивость	разброс и выбросы	std/IQR пропускной способности по повторам; число `simulation_error` + таймаутов
Автономность	вмешательства человека	отвеченные стендом запросы + настенное время, с логированием
Стоимость	использование LLM	`llm_calls`, оценка токенов, настенное время на генерацию

Правило агрегирования (воспроизводимое). По кейсу и методу сообщаемая величина — это **медиана по `ok`-повторам** этого метода от среднего по повтору MAPE; агрегат по кейсам — **медиана этих покейсовых величин**, с указанием `n` (никогда не объединение отдельных повторов). Загрузка исключена из агрегата MAPE и сообщается отдельно как APE метода A.

1.8 Повторности, сидирование, статистика

- `repetitions = 3` (независимые генерации LLM, свежие seed); точность эталона даёт `reference.replications = 10`, и она не зависит от стоимости LLM.
- Повторы никогда не отбрасываются ради улучшения чисел метода.
- Статистический план: знаково-ранговый Wilcoxon по кейсам для парных сравнений (пары по кейсу); при наличии A/B/C всех вместе — Friedman по каждому показателю
  1. апостериорный Nemenyi; KS по каждому показателю для размерности поведения;

α = 0.05, двусторонний. **При n = 5 кейсов эти тесты недостаточно мощные и сообщаются только как ориентировочные.**

1.9 Что сознательно не сравнивается

- Нет «управляемого» второго прохода (перегенерация с подсказкой от эталона) — только базовый.
- Мультифизический скрипт PyChrono метода C **измеряется, но никогда не исполняется**: PyChrono нет на хосте стенда, и скрипт моделирует твёрдые тела, а не дискретно-событийную семантику кейса. Фиксируются только размер, упоминания идентификаторов станков и импорт `pychrono`, и из него ничего не выводится.
- Поточная стоимость по токенам (только число вызовов и настенное время; `llm_tokens_estimate` равен null для всех трёх).

1.10 Почему SimBench нельзя было применить к A и B

Что такое SimBench. Это **собственный** бенчмарк метода C, лежащий в его репозитории (`benchmarks/simbench.py`, `benchmarks/README.md`, входы `benchmarks/llm_judge/llm_judge_input/simbench_input_turn[1-3]_v0.json`). Он реализует протокол статьи Wang J. et al., «SimBench: A Framework for Evaluating and Diagnosing LLM-Based Digital-Twin Generation for Multi-Physics Simulation» (IEEE Trans. AI; arXiv:2408.11987v2): три хода (Turn 1 Vague → Turn 2 Sharp → Turn 3 Complex) на задаче цифрового двойника непрерывной разливки стали, где модель должна выдать **скрипт PyChrono** (мультифизика твёрдых тел), а оценка — это рубрика J-LLM-as-a-judge (Completeness 40, Correctness 30, Code Quality 10, Efficiency 10, Error Handling 5, Visualization 5) плюс попарное сравнение «sllm против baseline» и Pass@1.

Почему он неприменим к A и B:

1. **Другой класс артефакта (другой движок).** SimBench оценивает скрипт **PyChrono** по критериям PyChrono API (`ChSystemNSC`, `AddBody`/`AddLink`, `ChLidarSensor`, `DoStepDynamics`, `chrono_irrlicht`). Метод A выдаёт сеть компонентов **FactorySimPy**, метод B — самодостаточную модель **SimPy** (построенную через pm4py из журнала событий). Ни один не производит PyChrono-код, поэтому у рубрики нет объекта: любой артефакт A или B провалил бы Completeness/Correctness/Visualization просто за «не тот фреймворк». Такой балл измерял бы формат вывода, а не методы.
2. **Другой предмет оценки: рубрика судьи против восстановления KPI.** Оценка SimBench — это суждение LLM-судьи и попарное предпочтение относительно baseline, то есть градиент **правдоподобия и качества кода** по тексту кода. Вопрос нашего сравнения численный: насколько throughput/WIP/энергия модели близки к сидированному SimPy-эталону. У SimBench нет ни эталонной установки, ни KPI, поэтому он не может ответить на вопрос о поведенческой точности ни для кого — ни для A и B, ни даже для C, чей прогон SimBench оценивает слот PyChrono, а не слот DES, о котором это сравнение.
3. **Другая задача и другие входы.** Входы SimBench — фиксированный сценарий разливки стали с текстами заданий и эталонными baseline-кодами на трёх нарастающих ходах. У A и B собственные входы (`prompt_A.txt` — проза с таблицей; `eventlog.csv`) и они **одношаговые** генераторы: нет диалога Turn 1/2/3, который можно было бы прогнать, и нет сталелитейного задания, которое можно им подать. Подсунуть им тексты про сталь означало бы сменить задачу, и мерилось бы уже не то, что мерит этот конвейер.
4. **Он заменил бы предмет исследования.** Наше сравнение держит набор кейсов (5 SimPy-эталонов) фиксированным и оценивает каждый метод против одной и той же истины; SimBench подменил бы набор кейсов одним сталелитейным сценарием, а численную истину — мнением судьи, то есть это был бы другой эксперимент.

Поэтому у SimBench нет строки для A и B: он остаётся внутренним бенчмарком C (оценивая внутри C не тот слот, который интересует это сравнение), а не альтернативным протоколом для всех трёх методов.

---

2. Метод A — FactoryFlow (кратко)

- **Что это.** Генератор «естественный язык → имитация». Получает `prompt_A.txt` — прозу плюс таблицу, описывающую ту же установку, которую проектировали авторы A, — и возвращает исходный код Python, задающий параметризованную сеть компонентов **FactorySimPy**.
- **Как его запускает стенд.** `from IM.AMG_with_langgraph import run_amg_wrapper`, с `cwd` в каталоге Streamlit `app/` и `sys.path`, расширенным `app/` и `app/src` (FactorySimPy не установлен через pip). Сгенерированный код выполняется в свежем подпроцессе за санирующим прелюдом (заглушка модуля `streamlit`, `MPLBACKEND=Agg`), а зонд, добавленный после модели, читает `Machine.stats` (`num_item_processed`, `total_time_spent_in_states`) плюс содержимое буфера и печатает одну меченую строку JSON. `generation_error` у этого метода часто является HTTP 402 (лимит кредита/максимума токенов), а не отказом моделирования; `simulation_error` — это выход сгенерированной модели с ненулевым кодом.
- **Что он умеет выражать.** Пропускную способность и загрузку по каждому станку. **WIP и энергия не входят в его контракт вывода**, поэтому эти столбцы у A пусты (`null`) по построению.
- **Известные оговорки измерения** (фиксируются, не исправляются):
  - **У готовности нет примитива в библиотеке.** В FactorySimPy нет понятия mtbf/mttr или отказа/ремонта, поэтому готовность приходится писать руками в сгенерированном коде. Два наблюдаемых исхода одного кейса показывают оба способа, как это незаметно проваливается: одна модель вовсе проигнорировала готовность и всё равно дала пропускную способность в пределах 0.2 % от эталона, другая выдумала API (`Machine(..., mtbf=…)`) и попала в `simulation_error`. **`ok` у A не означает, что готовность кейса была смоделирована** — читайте сгенерированный код, прежде чем приписывать KPI системе.
  - **Холодный старт.** Промпт A просит смоделировать `window_s` от t = 0, поэтому его окно измерения — `[0, 432000]` против эталонного `[21600, 453600]`: длительности совпадают, но A сохраняет 6-часовой переходный процесс и отбрасывает последние 6 ч. Существенно только для остаточной ошибки у уровня шума (пропускная способность куда менее чувствительна к переходному процессу, чем WIP, который A не сообщает).
- **Почему не SimBench.** SimBench оценивает скрипт PyChrono, тогда как A выдаёт сеть FactorySimPy; его рубрика оценивала бы фреймворк, а не метод (§1.10).

---

3. Метод B — LLM-DES-Modeling (кратко)

- **Что это.** Генератор «process mining → имитация». Потребляет `eventlog.csv` установки (`MachineName`, `StartTime`, `EndTime`, `ReasonCode` ∈ {Working, Idle, Stopped}, `EnergyConsumption` в кДж), вычисляет метрики pm4py и представление последовательности, затем просит у builder-LLM самодостаточную модель **SimPy**.
- **Как его запускает стенд.** В venv LLM-DES, повторяя headless-путь репозитория: `eventlog.load/preprocess`, `metrics.compute`, `to_sequence_text`, затем `ModelBuilder(client).build(blueprint_code, stations_md, sequence_text, buffers, defects, manual_note)`, где `aux_B.txt` поставляет буферы/дефекты. Адаптер передаёт blueprint и таблицу `stations_md`, выведенную из того же набора кейса. Исполнение переиспользует собственный раннер репозитория: модель запускается подпроцессом, а её контракт stdout разбирается (`Throughput = … parts/hour`, `WIP = … parts`, `Mean Energy Consumption per Part = … kWh/part`).
- **Горизонт (исправлено 2026-09-18).** Ранние прогоны B подавали фиксированную 7-дневную имитацию после 1 дня прогрева независимо от кейса, тогда как эталон измеряет 5 дней после 6 ч; **любой результат B, записанный до этого исправления, не сравним с `truth_kpis.json`.** Сейчас адаптер выводит `sim_time = warmup_s + window_s` и `warmup_seconds = warmup_s` из блока `reference` набора (значения из конфига — только запасной вариант). Исправление действует в прогоне, цитируемом в §5.
- **Что он умеет выражать.** Пропускную способность, WIP и энергию на деталь — но **не загрузку**, поэтому сравнение загрузки есть только у A.

- **Почему не SimBench.** SimBench оценивает скрипт PyChrono, тогда как B выдаёт самодостаточную модель SimPy после шага process mining; его рубрика измеряет правдоподобие кода по суждению судьи, а не восстановление KPI (§1.10).

---

4. Метод C — DigitalTwinBuilder и DES-этап внутри его DT-агента

4.1 Сервис и его агенты

C — не локальный инструмент: это сервис FastAPI, доступный по HTTP по URL из `dtb_repo/.env` (`API_URL`; в итоговом прогоне — http://176.119.159.227:8000). Агенты адресуются по индексу — **UI = 0, DB = 1, DT = 2**, — таково отображение в собственном репозитории метода. Конвейер стенда: `POST /sessions` → `POST /conversations` (с системным промптом самого C) → `POST /conversations/{id}/messages` → `POST /tasks` → опрос `GET /tasks/{id}`. Подача задачи **никогда не повторяется** (повтор выставил бы счёт модели дважды); чтения и опросы повторяются.

Сервис — это хост инференса без опубликованной карточки модели: личность агента и его собственные id модели **не записываются** в `environment.json`. Записываются ответчик на стороне стенда (`openai/gpt-5-mini`) и разбивка вызовов C по агентам.

4.2 От интервью к требованиям и схеме

1. **UI-агент (0)** — цикл интервью до тех пор, пока флаг `completed` в ответе не станет истинным или не будет достигнут `C.max_interview_turns`. Выход: **JSON требований** (топология линии, станции, буферы, поступления, датчики, `des_horizon`, …).
2. **DB-агент (1)** — разговор, засеянный `make_db_prompt(requirements)`, возвращает **DDL PostgreSQL** (`CREATE TABLE`/`INSERT`/индексы, представления опционально). С поправки к БД от 2026-09-21 ответ валидируется как **SQL**, а не как JSON: `sql_runner.check` только парсит (сервер не опрашивается) и требует хотя бы одного `CREATE TABLE`, отклоняя JSON-документ, неразбираемые операторы и отношения, использованные, но не созданные. Отклонённый ответ вызывает ход починки (`make_db_repair`, до `C.db_repair_attempts`).
3. **DT-агент (2)** — три слота разговора (ниже), которым подаются требования **и** схема.

4.3 Три слота DT-агента

DT-агент управляется через три слота, в точности как в собственном `streamlit-app.py` метода (те же промпты, тот же `conv_idx`, те же бюджеты токенов):

Слот	Системный промпт	Пользовательский промпт	Выход	`max_tokens`
0	`GenConf`	`make_gen_conf(requirements, db_schema)`	конфигурация двойника (JSON)	3000
1	`GenSim`	`make_gen_sim(requirements, db_schema)`	мультифизический скрипт **PyChrono**	3000
2	`GenDES`	`make_gen_des(requirements, db_schema)`	**дискретно-событийная модель SimPy**	**8000**

Все промпты `exec`-ятся во время выполнения из `dtb_repo/src/digital_twin_builder/prompts/*.py`, поэтому правка промпта в репозитории C распространяется на следующем прогоне без изменений в стенде. Слот 1 захватывается и измеряется, но никогда не исполняется (§1.9).

4.4 Слот DES: адаптируй эталонную программу, а не пиши свою

Слот 2 просит программу SimPy, печатающую **тот же трёхстрочный контракт KPI, что печатает B** — через тот же парсер (`parse_b_stdout`), тот же горизонт и то же определение измерения — именно это делает числа C и B напрямую сравнимыми.

Тяжёлую работу делают два свойства промпта:

- **Эталонный blueprint (`DES_BLUEPRINT`).** `GenDES` встраивает проверенную программу SimPy: источник → станции → буферы → сток, с параметризованным блоком `PARAMETERS`, `STATIONS`, `BUFFERS`, `INTER_ARRIVAL_S`, `WARMUP_S`, `WINDOW_S`, `REPLICATIONS`. Промпт формулирует задачу как **адаптировать, а не писать** — поменять значения параметров (и длину списков станций/буферов) и не трогать модель и блок KPI. Поэтому блок KPI **наследуется**, а не пишется.

- **Арифметика KPI в blueprint точна по построению.** WIP и потребление мощности интегрируются `Ledger` на **гранулярности событий** (каждое событие, которое двигает деталь или меняет состояние станции, вызывает `sync` **до** изменения), поэтому ничего не дискретизируется и состояние короче любого интервала выборки всё равно вносит ровно свою длительность. Интервалы до границы прогрева отбрасываются, что делает прогрев бесплатным. Пропускная способность делит детали, покинувшие последнюю станцию, **на** `window_s / 3600` — ошибка «счётчик против скорости», ломавшая ранние программы, структурно исключена. Повторения сидируются, а среднее по каждому KPI печатает собственный `main()` blueprint.
- Промпт также задаёт определения KPI и **самопроверку**: последовательная линия не может превысить `3600 / max(processing_time_s)` деталей в час, а энергия на деталь не может превысить потребление одной занятой детали, делённое на худшую готовность.

Требования, подаваемые в слот, несут инъектированный `des_horizon = {warmup_s, window_s, replications}` из набора кейса (например, case_0001: 21600 / 432000 / 10).

4.5 Исполнение, приёмка и цикл «исполни-и-почини»

Стенд не доверяет тому, что напечатанные числа пришли из той линии, которая была задана. Слот DES запускает собственный `des_runner.py` метода C:

- **`execute`** — записывает программу, запускает её под `C.des_python` (`comparison/.venv-des/bin/python`, интерпретатор с поддержкой SimPy), с `C.des_exec_timeout_s = 900`.
- **`parse_kpis`** — извлекает три контрактные строки из stdout.
- **`verify`** — принимает только если программа отработала чисто, **и** напечатала контракт, **и** числа возможны для объявленной линии. Семантические ворота — это `_serial_bounds` + `_implausible_kpis`, выводящие границы из **собственных параметров требований** (никогда из эталона кейса):
  - потолок пропускной способности `3600 / max(processing_time_s) × 1.05`;
  - нижняя граница `min(3600/max_processing_time_s, 3600/inter_arrival_time_s) × min(availability) × 0.5`;
  - потолок энергии `Σ(power_working_kw × processing_time_s / 3600) / min(availability) × 1.10`;
  - WIP и энергия не могут быть равны 0, пока производятся детали (`_all_zero_implausible`, управляется `C.des_reject_all_zero`).

  Непоследовательная топология не ограничивается; отсутствующая `topology` читается как последовательная; граница не выдумывается при отсутствии параметра; потолок энергии снимается, когда станция в простое потребляет больше, чем в работе. Запасы (1.05, 0.5, 1.10) взяты из отступа, который эталон пяти референсных линий держит по каждой границе. Искажённые формы требований читаются как **отсутствие границы**, а не вызывают исключение, потому что ворота прогоняются на каждом повторении.
- **`generate_with_repair`** — вердикт, отличный от `ok` (падение, таймаут, нет контракта или нарушение границы), возвращается **дословно** вместе с предыдущей программой как следующий ход (`make_gen_des_repair`), до `C.des_repair_attempts` = 3 (не более 4 программ). `des_stage` и поля KPI описывают **последнюю** программу; `des_gave_up_reason` равен null, когда последняя программа принята.

Собственное приложение C использует тот же движок (`streamlit-app.py::dt_generate_des`), поэтому стенд и продукт согласованы по построению.

4.6 Почему измерение принадлежит C, а не стенду

Поскольку стенд воспроизводит поток управления C, а не вызывает приложение Streamlit, ворота БД, ворота DES и циклы починки пришлось **зеркалировать** в `adapters/method_c_adapter.py`, чтобы исправления дошли до измерения. Они не перепечатаны: `_des_gate()` exec-ит `des_runner.py` метода C, а `_sql_gate()` exec-ит его `sql_runner.py`, и как усиленный системный промпт, так и пользовательские промпты починки загружаются из `prompts/` C. Следствия:

- Повтор не может пройти здесь проверку, которую он провалил бы в собственном приложении C.
- Правка промпта или ворот в репозитории C распространяется без изменений в стенде.

- Записанные метки прогонов это отражают: прогон после правки ворот — это **другой метод**, чем прогон до неё, и их нельзя объединять.

Семантика статусов не сдвигается от этих поправок. Вердикт ворот — это **фиксируемое свойство** (`db_stage`, `des_stage`), никогда не вход статуса повтора: `derive_rep_status` читает заметки DB/DES только для таймаута или недоступной конечной точки. Поправке не позволено улучшать вид C переклассификацией повтора, поэтому, например, программа `0/0/0` всё равно читается как `des_stage: ok` и её надо отделять от настоящего измерения, читая `des_gave_up_reason` / `des_programs` / `des_repairs_used`.

4.7 Что дал DES-этап в итоговом прогоне

В `20260921_073238_post-amendment`:

- **Все 14 `ok`-повторов произвели несущую контракт модель DES** — `des_stage = ok`, `des_programs = 1`, `des_repairs_used = 0`, `des_gave_up_reason = null`, и каждая сгенерированная программа несёт собственные классы blueprint (`class Ledger`, `class Sink` присутствуют во всех 14 `artifacts/dtb_des.py`). **Ни один раунд починки не был потрачен нигде в прогоне**, что и есть задуманный эффект blueprint: программы с первой попытки наследуют правильную арифметику KPI, а не пишут её. (Для контраста, в прогоне до blueprint `20260920_064710_des-repair` `class Ledger` есть в **0 из 10** DES-артефактов.)
- Единственный повтор без модели — это таймаут опроса слота DES, а не отказ ворот: слот не вернулся в пределах своего бюджета 900 с.
- Само исполнение DES дёшево: `des_exec_elapsed_total_s ≈ 1.0 с` на повтор (SimPy на гранулярности событий) против медианного времени генерации ≈ 426 с, то есть стоимость сидит в LLM-агентах, а не в прогоне модели.

---

5. Итоговый прогон по каждому методу

5.1 Происхождение — три прогона, а не один

Используется последний **реальный** прогон по каждому методу. Ни одного реального прогона со всеми тремя методами на текущем коде не существует, а два новейших каталога прогонов (`20260922_110432`, `20260922_111306`) — это `mode: mock` прогоны «сантехники» и **не результаты**. Поэтому три столбца идут из трёх разных прогонов; они сравнимы, поскольку наборы кейсов, эталонный движок, контракт вывода и код оценки идентичны, но это **не единый эксперимент** и сообщать их нужно поблочно по методам.

Метод	Каталог прогона	Метка	Дата	Режим	Повторы	Кейсы	LLM метода
A	`runs/20260918_043936`	—	2026-09-18	real	3	5	FactoryFlow, `google/gemini-3.1-pro-preview` через OpenRouter
B	`runs/20260918_163803`	—	2026-09-18	real	3	5	LLM-DES-Modeling, `gpt-5.1` builder/inspector через OpenRouter
C	`runs/20260921_073238_post-amendment`	`post-amendment`	2026-09-21	real	3	5	Сервис DigitalTwinBuilder (http://176.119.159.227:8000); ответчик стенда `openai/gpt-5-mini`

Почему именно эти три:

- **A** — это единственный реальный прогон, в котором A дал сравнимые KPI (`20260918_043936`, общий с B). Более поздний прогон только для A (`20260918_162954`) покрывает лишь case_0005 и вернул `simulation_error` во всех 3 повторах.
- **B** — его числа валидны только **после** исправления горизонта (§3), и `20260918_163803` — первый пост-фикс прогон B по всем 5 кейсам. Все более ранние прогоны B цитировать нельзя.
- **C** — прогон, несущий правки DES от 2026-09-20/21 (цикл «исполни-и-почини» и `DES_BLUEPRINT` / семантические ворота приёмки, §4.4–§4.5); каждая из его 14 сгенерированных DES-про-

грамм содержит собственные классы blueprint, поэтому цитируемые числа — это числа метода после blueprint.

Единственный не-ok повтор прогона C — таймаут опроса слота DES на 900 с (case_0005 повтор 2: слот не вернулся, поэтому модель не была произведена, и повтор — timeout). По §1.3.3 он фиксируется, а не переигрывается.

5.2 Сводная таблица — итоговый прогон по каждому методу

Метод	Прогон	Валидность (ok / 15)	Кейсы с KPI (n)	Медиана MAPE по кейсам
A	20260918_043936	**4 / 15**	**2 / 5**	**3.6 %** (n = 2)
B	20260918_163803	**14 / 15**	**5 / 5**	**31.6 %** (n = 5)
C	20260921_073238_post-amendment	**14 / 15**	**5 / 5**	**1.5 %** (n = 5)

Не-ok повторы и их причина:

- **A** — 8 simulation_error (сгенерированная модель вышла с кодом 1: 3× case_0001, 2× case_0002, 3× case_0003); 3 generation_error (case_0005 ×3 — OpenRouter **HTTP 402**, лимит кредита/максимума токенов, а не отказ моделирования).
- **B** — 1 simulation_error (case_0002 повтор 2, сгенерированная модель вышла с кодом 1).
- **C** — 1 timeout (case_0005 повтор 2 — таймаут опроса слота DES на 900 с; модель не произведена).

Медиана MAPE по показателям (медиана по кейсам от покейсовой медианы; – = метод не может выразить показатель):

Метод	Пропускная способность	WIP	Энергия на деталь
A	3.6 % (n = 2)	—	—
B	24.7 % (n = 5)	39.3 % (n = 5)	22.6 % (n = 5)
C	**0.5 %** (n = 5)	**1.8 %** (n = 5)	**0.0 %** (n = 5)

Дополнительные структурные/поведенческие факты из тех же прогонов:

- **A** — APE загрузки 0.3 % (совпало 3/3 станка, case_0002) и 7.1 % (5/5 станков, case_0004). Покейсовые медианы MAPE, стоящие за 3.6 %, — это 0.17 % (case_0002, 1 ok-повтор) и 7.03 % (case_0004, 3 ok-повтора).
- **B** — 1 из 42 ячеек KPI попадает внутрь собственного относительного std эталона; 3 из 15 повторов встают в дедлок и печатают Throughput = 0.00 при ненулевом WIP; один повтор падает outright на simpy.Store(env, capacity=None).
- **C** — структурное восстановление по всем 15 повторам, каждый несёт свидетельства этапов (69 слотов станков): станки упомянуты **69/69**, время обработки восстановлено **69/69**, готовность восстановлена **41/69**, ёмкости буферов **69/69**, интенсивность поступлений найдена **15/15 повторов**.

5.3 Таблица по кейсам

ok/3 = число ok-повторов для этого кейса; процент — медиана MAPE кейса по его ok-повторам (величина, которую агрегирует §1.7). – = нет KPI от этого метода для этого кейса.

Кейс	Эталон T / WIP / E	A ok/3	A медиана MAPE	B ok/3	B медиана MAPE	C ok/3	C медиана MAPE
case_0001_serial_line	19.868 / 40.387 / 0.9892	0/3	—	3/3	31.60 %	3/3	**1.45 %**
case_0002_saturated_shop_tight	32.620 / 27.917 / 0.3525	1/3	0.17 %	2/3	17.95 %	3/3	**0.72 %**
case_0003_starved_six_line	11.991 / 5.251 / 2.3537	0/3	—	3/3	230.13 %	3/3	**9.51 %**
case_0004_failure_heavy_line	22.401 / 7.289 / 0.8735	3/3	7.03 %	3/3	38.28 %	3/3	**4.24 %**
case_0005_high_rate_line	44.991 / 38.882 / 0.3354	0/3	—	3/3	30.18 %	2/3	**0.43 %**
агрегат		**4/15**	**3.6 %** (n=2)	**14/15**	**31.6 %** (n=5)	**14/15**	**1.5 %** (n=5)

Чтение по кейсам: там, где C произвёл модель, он укладывается менее чем в 1.5 % на трёх из пяти кейсов; наибольшее отклонение — case_0003 (9.5 %, из-за перелёта WIP на +27.8 % в двух из трёх повторов), следующим идёт case_0004 (4.2 %), где остаток — это WIP и пропускная

способность на линии с частыми отказами. Собственная валидация blueprint приписывает это направление ошибки **модели отказов эталонного движка** (поломка взводится только пока станок ожидает, поэтому постоянно занятый станок никогда не отказывает) — модель B отклоняется в ту же сторону (case_0004: 16.8 / 12.78 против эталона 22.4 / 7.29). Это общий потолок, который надо читать рядом с ошибкой любого метода, а не как дефект только C.

5.4 Стоимость и автономность (те же прогоны)

Метод	`llm_calls` (всего / медиана на повтор)	Разбивка C по агентам	Подачи агента	Время генерации, медиана (сумма)	Время имитации, медиана
A	24 / 2	—	—	55.7 с (663 с)	0.1 с
B	30 / 2	—	—	56.5 с (820 с)	16.2 с
C	81 / 5	ui 23 · db 14 · dt 44	83 попытки на 81 ответ	425.9 с (7 899 с)	1.0 с

Чтение: C покупает свою точность примерно втрое большим числом вызовов LLM и в семь раз большим настенным временем, чем A/B, и его расходы лежат внутри его собственного конвейера (один DT-агент — это 44 из 81 вызова). Стоимость автономности в этом прогоне минимальна — один ход интервью на повтор, во всех 15 повторах, — и, после поправки, ноль раундов починки.

5.5 Структурное восстановление (только C)

Размерность	Итоговый прогон
упомянуты идентификаторы станков (против эталона)	69 / 69
восстановлено время обработки	69 / 69 (100 %)
восстановлена готовность	41 / 69 (59.4 %)
восстановлены ёмкости буферов	69 / 69
найдена интенсивность поступлений	15 / 15 повторов
слоты DT ответили (`dt_slots_ok / total`)	3 / 3 на 14 из 15 повторов (2 / 3 на повторе с таймаутом)
идентификаторы станков в скрипте PyChrono	5/5, 3/3, 6/6, 5/5, 4/4 по кейсам

Параметр считается восстановленным, только когда C указывает число **под ключом, называющим это понятие**, и оно равно эталону; число, которое C фактически заявил, фиксируется рядом с вердиктом (`*_claimed`), поэтому выдуманное значение видно, а не схлопывается в булево. Готовность, заявленная в процентах, принимается против эталонной доли; значение вида 95 против 0.92 — ни то ни другое.

---

6. Как читать таблицу — угрозы валидности

1. **Это не единый объединённый прогон.** Три столбца приходят из трёх прогонов трёх дат (§5.1). Внутри каждого столбца сравнение чистое (те же наборы, тот же эталон, та же оценка); **между** столбцами это сравнение результатов методов, а не одного одновременного трёхармного эксперимента. Прогон публикационного уровня был бы единым `python run_comparison.py --mode real --repetitions 3` со всеми тремя методами на текущем коде.
2. **Неравные знаменатели.** 3.6 % у A — это медиана по **2 кейсам** (A дал KPI лишь в 4 из 15 повторов), тогда как у B и C — по 5. Поэтому сводное число A стоит не на той же почве, что у остальных, и его низкая доля успеха — более важный факт о нём. Любой рейтинг A/B/C обязан нести `n`.
3. **Все три отказа A на case_0005 — инфраструктурные** (OpenRouter HTTP 402 — лимит кредита/максимума токенов), а не поведение моделирования. Они остаются в таблице как `generation_error`, потому что §1.3.3 запрещает перебросы; они не являются свидетельством о методе.
4. **Поведенческие числа C условны по слоту DES.** Пропускная способность, WIP и энергия существуют, только когда этот слот даёт исполнимую, несущую контракт модель; Валидность, Автономность, Стоимость и структурное восстановление — это размерности, сравнимые по **каждому** повтору C. Поведенческие числа C здесь — это 14 из 15 повторов, а в меньшем/деградировавшем прогоне хоста выход может обвалиться (в `runs/20260919_052152` он был 0/3).
5. **DES-числа C идут через тот же парсер и горизонт, что у B**, но это не тот же генератор: сравнимость **измерения** точна, сравнимость **задачи** — суть исследования.
6. **Скрипт PyChrono метода C никогда не исполняется**, поэтому никакого мультифизического утверждения о C не поддерживается — только то, что слот отработал и что он выдал.
7. **У эталонного движка ограниченная модель отказов** (поломки только пока станок свободен), и это доминирующая общая ошибка на `case_0004`; все методы оцениваются против одного и того же ограниченного эталона, поэтому это общий потолок, а не смещение по методу.
8. **n = 5 кейсов, R = 3.** Wilcoxon/Friedman недостаточно мощные; результаты ориентировочные. Выборка LLM сознательно не пришпилена температурой, поэтому разброс между повторами — измеряемое свойство.
9. **Сервис — чёрный ящик**: личность агентов C и модели не опубликованы и не записаны, поэтому прогон нельзя воспроизвести **побитово** против зафиксированной модели; `api_url` прогона и число вызовов по агентам записаны в каждом `result.json`.

---

7. Воспроизводимость

```
cd /ssd/projects/dt-eval/comparison

python run_comparison.py --mode mock                 # сантехника, без затрат
python run_comparison.py --mode real --repetitions 3  # полный трёхметодный дизайн
python report.py runs/<timestamp>                    # отрендерить report.md
```

Интерпретаторы/рабочие каталоги, настройки по методам и выбор кейс/метод/повтор берутся из `config.yaml`; ключ OpenRouter лежит в `.env` (экспортированная переменная оболочки его перекрывает). Оркестратор сам запускает `casegen.py`, когда порождаемые артефакты набора отсутствуют.

На каждую генерацию `runs/<timestamp>/<case>/<method>/rep<r>/` содержит `result.json`, сгенерированную модель, stdout/stderr исполнения и (для C) каталог `artifacts/` с JSON требований, ответом схемы (`dtb_schema.sql`) и его JSON-обёрткой, конфигурацией двойника, скриптом PyChrono, DES-программой и `interview.log` со всеми ходами и всеми вердиктами ворот. Архив для публикации: наборы кейсов, `runs/`, требования venv, входные промпты и список моделей из `RUN_WITH_OPENROUTER.md`.

Цитирование трёх прогонов

- A: `comparison/runs/20260918_043936` (столбец A; в том же каталоге лежат и результаты B до исправления горизонта, они перекрыты — для B использовать `20260918_163803`).
- B: `comparison/runs/20260918_163803`.
- C: `comparison/runs/20260921_073238_post-amendment`.